

Coding Exercise — Todo/Task-List App with a Small LLM Feature

1 Goal

Build a small but well-structured Todo/Task-List application **plus one lightweight AI feature backed by a large-language model (LLM)**.

Publish the code in a public GitHub repository **before our interview**. During the meeting we'll review the project together and discuss your technical decisions.

2 Functional requirements

Core features

- Add, edit, delete and mark-as-done tasks.
- Data persistence can be local (file, SQLite, or in-memory); no user authentication required.

LLM feature — choose one of these ideas or invent a similarly small alternative

- **Natural-language** → **Task**: e.g. “remind me to submit taxes next Monday at noon” → title + date.
- **Smart labeling**: the LLM assigns priority/labels based on the task description.
- **Bulk summariser**: the LLM produces a daily summary of pending or completed tasks.

(Keep the scope tiny and the UX simple.)

Tech stack (pick what you prefer)

- **Frontend**: React (plain React, Vite/CRA, or Next.js).
- **Backend**: Python (FastAPI, Flask, or Django REST).
- Feel free to use TypeScript, a CSS framework, testing tools, linters, Docker, etc., if that reflects how you normally work.

Scope

Focus on code clarity and project structure rather than visual polish or production-grade scalability.

3 What to deliver

1. **GitHub repository** containing
 - All source code and configuration files (add an `.env.example` for API keys).
 - A *readable* commit history that shows how the project evolved (don't squash everything away).
 - A **README.md** that explains
 - how to run the app locally (one-command start is ideal);
 - the key design or architectural choices, especially around the LLM (prompt strategy, error handling, fallbacks);
 - how to run tests, if present.
 2. **Repository link** emailed to me **≥ 24 hours before the interview** so there's time to review it.
-

4 Evaluation criteria

Area	What we look for
Code quality	Clear, idiomatic, well-documented code; meaningful commit messages.
Project structure	Logical file/folder layout; clean separation of concerns; easy navigation.
LLM integration	Sensible use-case, safe prompt handling, graceful fallback if the model/API is unavailable.
Correctness	Core and AI features work as described; no blocking bugs.
Testing (<i>bonus</i>)	Unit/integration tests, including at least one around the AI logic (mocked response is fine).
Developer experience	Straightforward setup; helpful tooling (Docker, Makefile, linters) where relevant.
Communication	README and comments that justify key decisions and highlight trade-offs.

5 Interview agenda (30–45 min)

1. **Demo** — you run the app locally and show the LLM feature in action.
 2. **Code tour** — we navigate the codebase together.
 3. **Discussion** — deeper questions about your implementation, alternative designs, and next steps.
-

Tips

- Keep the AI portion *tiny but real*; even a single well-designed endpoint is enough.
- Provide mock or fallback data so the app still runs without an API key.
- Finished is better than perfect — aim for a small slice done well.
- Rehearse the demo; troubleshooting uses interview time.

Questions? Need an extension?

Just let us know; We're happy to help.